

ARM 体系架构 · 完整总结

NXP i.MX6ULL | Cortex-A7 内核 | ARMv7 指令集 | 32 位处理器

一、指令集架构：RISC 与 CISC

对比项	RISC（精简指令集）	CISC（复杂指令集）
全称	Reduced Instruction Set Computer	Complex Instruction Set Computer
设计理念	只保留最常用的简单指令，复杂操作靠组合完成	一条指令完成复杂操作，指令功能强大
指令特点	指令简单、长度固定、执行快	指令复杂、长度不固定、译码慢
芯片特征	简单、成本低、体积小、功耗低	复杂、成本高、体积大、功耗大
代表架构	ARM、RISC-V	x86
典型应用	手机、平板、嵌入式、物联网	PC、服务器、高性能计算

28效应：80% 的程序只用到 20% 的指令。RISC

精简指令集只保留高频指令，用流水线提升吞吐率，用编译器组合实现复杂功能。

ARM 处理器三大系列

系列	定位	典型内核	应用场景
Cortex-A	Application 高端应用	A7、A8、A9、A15、A97	消费电子：手机、平板、智能电视
Cortex-R	Real-time 实时性	R4、R5、R7	汽车、硬盘控制器、实时控制系统
Cortex-M	Micro 微控制	M0、M3、M4、M7	物联网、传感器、STM32 系列

- STM32F103 → Cortex-M3（ST 意法半导体）
- Samsung Galaxy Note → Cortex-A8
- Exynos 4412 → Cortex-A9
- i.MX6ULL → Cortex-A7（本课程目标芯片）

二、ARM 三级流水线结构

ARM7 采用三级流水线，同一时刻三条指令并发执行：

阶段	操作	说明
取指（Fetch）	PC 从存储器取指令	PC 指向当前要取的指令地址
译码（Decode）	译码器解析指令含义	确定操作类型、操作数、目标寄存器
执行（Execute）	ALU 执行运算	算术逻辑单元完成实际计算并写回结果

核心理念：取指 → 译码 → 执行 三条指令在同一时刻重叠执行，吞吐率提升约 3 倍。

三、存储架构

3.1 冯·诺依曼架构 vs 哈佛架构

架构	特点	总线	优劣
冯·诺依曼	指令和数据共享同一块存储空间	单总线	成本低，但取指和取数据不能同时进行，速度受限
哈佛架构	指令和数据拥有独立的存储空间	双总线	取指和取数据可同时进行，速度快；绝大多数现代计算机采用

3.2 NOR Flash vs NAND Flash

对比项	NOR Flash	NAND Flash
寻址方式	每个字节都可独立寻址，有地址和数据总线	块设备，以 512 byte 为单位访问，无独立地址/数据总线
随机访问	可像 RAM 一样随机读写，支持 XIP（芯片内执行）	不支持随机访问，需先读到 RAM 再执行
容量	小（1~32 MB）	大（可达数 GB）
成本	高	低
写入速度	慢	快
可靠性	高，位翻转少	低，需要 ECC 校验
典型应用	单片机（存代码，直接在 Flash 内执行）	SOC / 电脑（大容量数据存储，如 EMMC）

3.3 大端存储与小端存储

方式	规则	示例（0x12345678 存于 0x100~0x103）	应用
大端存储	低地址存高数据位，高地址存低数据位	0x100: 12 0x101: 34 0x102: 56 0x103: 78	网络传输（网络字节序）
小端存储	低地址存低数据位，高地址存高数据位	0x100: 78 0x101: 56 0x102: 34 0x103: 12	SOC、ARM 处理器（默认）

四、寄存器体系

4.1 通用寄存器（16 个）

Cortex-A7 内核共有 37 个寄存器，其中 16 个为通用寄存器（R0~R15），可被程序员直接操作：

寄存器	别名	用途
R0~R3	-	通用数据寄存器，函数传参（a, b, a+b 等临时数据）

寄存器	别名	用途
R4~R11	-	通用数据寄存器，保存局部变量
R12	IP	Intra-Procedure scratch，函数调用中间暂存
R13	SP	Stack Pointer 栈指针寄存器，指向栈顶，管理栈区空间
R14	LR	Link Register 链接寄存器，保存函数/异常返回地址
R15	PC	Program Counter 程序计数器，指向当前执行指令的下一条

- R0~R12 为纯数据寄存器，R13~R15 有专用别名和特殊功能
- ALU（算术逻辑单元）负责运算，通用寄存器存放运算数据

4.2 特殊寄存器

寄存器	全称	作用详解
CPSR	Current Program Status Register	存放程序运行状态：N（负数标志）、Z（零标志）、C（进位标志）、V（溢出标志）、I/F（中断禁止位）、T（ARM/Thumb 状态）、M[4:0]（工作模式位）
SPSR	Saved Program Status Register	异常发生时硬件自动将 CPSR 备份到 SPSR；异常返回时从 SPSR 恢复 CPSR，保证被中断程序状态不丢失

4.3 栈操作原理

栈遵循先进后出（LIFO）原则，由 SP 寄存器管理：

操作	SP 变化	说明
Push	SP 值减小（向低地址生长）	将数据存入 SP 指向的新位置
弹栈（Pop）	SP 值增大（向高地址移动）	从 SP 指向位置取出数据，SP 指向新栈顶

函数调用时：压栈保存 LR（返回地址）和局部变量 → 执行函数体 → 弹栈恢复 LR → BX LR 返回调用处。

五、MMU 与 Cache

5.1 MMU（内存管理单元）

MMU: Memory Management Unit, 将程序中的虚拟地址空间映射到实际物理地址空间，提高物理内存利用率。

- 每个进程拥有独立 4GB 虚拟地址空间，通过 MMU 页表映射到物理 RAM
- 关闭 MMU：虚拟地址 = 物理地址（裸机开发模式）
- 开启 MMU：程序访问虚拟地址 → MMU 翻译 → 实际物理地址
- i.MX6ULL：4GB 虚拟空间，512MB DDR3 物理内存

5.2 Cache（高速缓存）

Cache 位于 CPU 和内存之间，解决CPU 速度远快于内存的问题：

类型	全称	缓存对象	说明
ICACHE	Instruction Cache	指令	CPU 取指令时先查 ICACHE，命中则直接取；未命中则从内存加载
DCACHE	Data Cache	数据	CPU 读写数据时先查 DCACHE，命中则直接操作；未命中则从内存加载

ICACHE 对性能影响大：裸机开发中通常打开 ICACHE、关闭 DCACHE（避免数据一致性问题）。

六、总线架构：AHB 与 APB

SoC 内部内核与外设通过总线连接通信，分两种总线架构：

架构	说明		
单总线	内核和外设之间通过一组线连接通信（冯·诺依曼架构）		
双总线	内核和外设之间通过两组线通信：高速总线 + 低速总线（哈佛架构）		

总线	全称	特点	连接外设举例
AHB	Advanced High-performance Bus	高速、高带宽	USB、网口、RAM（DDR3）
APB	Advanced Peripheral Bus	低速、低功耗	Timer、GPIO、UART

设计逻辑：高速外设接 AHB 直接与内核高速通信；低速外设接 APB 通过桥接与 AHB 连接，降低功耗和设计复杂度。

七、ARM 处理器 7 种工作模式

模式	英文	触发条件	特权	说明
用户模式	User	正常程序运行	非特权	不能直接访问系统资源，不能切换模式
系统模式	System	操作系统内核任务	特权	与 User 共用寄存器，但具有特权访问能力
管理模式	Supervisor (SVC)	复位 / SWI 软中断	特权	系统启动时进入，内核运行在此模式
快中断模式	FIQ	高速中断请求	特权	R8~R12
普通中断模式	IRQ	普通外部中断	特权	一般中断处理
中止模式	Abort	非法地址访问	特权	数据访问中止 / 预取指令中止
未定义模式	Undefined	未识别指令	特权	遇到 CPU 无法解码的指令

- 非特权模式只有 User；其余 6 种均为特权模式，可访问全部系统资源、修改 CPSR 切换模式

- 异常模式有独立的 R13（SP）和 R14（LR），防止异常处理破坏用户栈和返回地址
- FIQ 专属：额外拥有 R8~R12 五个独立寄存器，无需压栈保存，响应更快

八、异常处理体系

8.1 异常向量与向量表

异常向量：跳转到异常服务函数的地址标号。异常向量表：存放跳转到各异常服务函数指令的数组，每个异常在固定偏移位置。

地址	异常类型	优先级	进入的模式	用途
0x00	Reset（复位）	最高	SVC	上电或硬件复位，系统初始化入口
0x04	Undefined（未定义指令）	6	Undefined	CPU 遇到无法解码的指令
0x08	SWI（软中断）	6	SVC	程序主动调用 SWI 指令请求系统服务
0x0C	Prefetch Abort（预取中止）	5	Abort	取指令时地址非法或权限不足
0x10	Data Abort（数据中止）	2	Abort	数据读写时地址非法或权限不足
0x14	Reserved（保留）	–	–	未使用
0x18	IRQ（普通中断）	4	IRQ	外部设备中断请求
0x1C	FIQ（快速中断）	3	FIQ	高速中断，响应速度最快

中断向量（51单片机概念）= 标号，根据向量找到中断服务函数。异常向量（ARM概念）= 跳转到异常服务函数指令的地址。两者本质类似，表述不同。

向量表代码示例：

```
//
0x00: b reset_handler //      → SVC
0x04: b undefine_handler //    → Undefined
0x08: b swi_handler //      → SVC
0x0C: b prefetch_abort_handler //  → Abort
0x10: b data_abort_handler //    → Abort
0x14: b . //
0x18: b irq_handler // IRQ → IRQ
0x1C: b fiq_handler // FIQ → FIQ

//      51
void (*p[5])(void) = {fn1, fn2, fn3, fn4, fn5};
```

8.2 异常处理完整流程

当异常发生时，硬件自动完成前 4 步，软件完成后 2 步：

阶段	步骤	操作	由谁执行
硬件自动	①	将 CPSR 值备份到对应模式的 SPSR	硬件
硬件自动	②	CPSR M[4:0] I=1/F=1 T=0 ARM	硬件
硬件自动	③	LR R14	硬件
硬件自动	④	PC 指向异常向量表中对应入口地址	硬件
软件编写	⑤	执行异常服务函数（压栈保存寄存器 → 处理异常 → 弹栈恢复寄存器）	程序员
软件编写	⑥	异常返回：SPSR → CPSR（恢复状态），LR → PC（恢复执行位置）	程序员

关键理解：硬件只负责"切入"（保存+跳转），程序员负责"处理+切回"（服务函数+恢复）。异常服务函数必须压栈保存通用寄存器，否则会破坏被中断程序的数据。

九、移位操作补充

操作	英文	规则	示例（1011 0010 右移 1 位）
逻辑右移	LSR	最高位补 0，不考虑符号位（无符号数）	0101 1001
算术右移	ASR	最高位补符号位，保留正负（有符号数）	1101 1001
循环右移	ROR	移出的最低位循环补到最高位	0101 1001 1 → 原最低位的0补到最高位

十、芯片内核版本对照表

芯片	内核	指令集版本	位数	厂商	典型应用
i.MX6ULL	Cortex-A7	ARMv7	32 bit	NXP 恩智浦	嵌入式Linux开发板
S3C2440	ARM920T	ARMv4TE	32 bit	Samsung 三星	早期嵌入式教学板
Exynos 4412	Cortex-A9	ARMv7	32 bit	Samsung 三星	Galaxy Note 手机
STM32F103	Cortex-M3	ARMv7-M	32 bit	ST 意法半导体	微控制器/物联网

本课程目标：i.MX6ULL（Cortex-A7 / ARMv7），4GB 虚拟地址空间 + 512MB DDR3 物理内存 + EMMC 存储。

十一、i.MX6ULL 芯片资源概览

资源	规格
内核	Cortex-A7, ARMv7 指令集, 32 位

资源	规格
虚拟地址空间	4GB（通过 MMU 映射）
物理 RAM	512MB DDR3
存储	EMMC（NAND Flash 类型）
内部 SRAM	iram（片上小容量高速 RAM）
外设	GPIO、Timer、UART、USB、网口、RAM控制器
总线	AHB（高速：USB/网口/RAM）+ APB（低速：Timer/GPIO/UART）
其他	OSC（时钟源）、RESET（复位）、VCC/GND（供电）